Integración de Mensajes MQTT en UruTurn!

Este documento detalla la arquitectura y el flujo de integración entre el protocolo MQTT y la plataforma **UruTurn**. El broker actúa exclusivamente como capa de transporte de eventos; **la API REST es la única fuente de verdad** para las reglas de negocio y la persistencia de datos.

Tanto el microservicio generador como el consumidor operan de forma aislada a la base de datos MySQL. La herramienta phpMyAdmin se mantiene únicamente con fines de administración manual externa.

Flujo de Procesamiento

```
None
Generador
   ├─ GET /personal ──────────────────────────────> API ──> MySQL
   └─ Publica solicitudes ──> Mosquitto ──> Consumidor
                                                  |
                               POST /reservas <──┘ API ──> MySQL
```

1. **Consulta inicial**: El generador consulta a la API el listado de profesionales activos (`GET /personal`) para seleccionar el destino del turno. El atributo `id` incluido en el mensaje MQTT identifica únicamente la publicación del evento; la API será la encargada de asignar el identificador definitivo de la reserva.
2. **Distribución del evento**: El broker Mosquitto recibe y distribuye el mensaje a través del tópico `turnos/solicitudes`.
3. **Traducción y consumo**: El servicio consumidor transforma el objeto `turno` recibido al contrato estándar de la API HTTP y lo envía mediante `POST /reservas`.
4. **Validación e ingesta**: La API valida los datos recibidos y sus referencias. Si la solicitud es válida, asigna un ID de reserva unívoco y registra el cliente junto a la solicitud con el estado **PENDIENTE**.
5. **Procesamiento asíncrono**: El worker de la API procesa de forma asíncrona las solicitudes en estado pendiente, confirmando o rechazando cada agenda según las reglas de negocio globales.

La API REST no contiene lógica, rutas ni acoplamiento técnico con el protocolo MQTT. Desconoce el origen de los tópicos o estructuras de mensajería; únicamente procesa solicitudes REST estandarizadas como cualquier otro cliente.

Formato de Mensajes y Mapeo

Estructura JSON (Mensaje MQTT)

```
None
{
  "status": "turno_creado",
  "fechaHora": "2026-11-05T12:00:00",
  "turno": {
    "id": 123456,
    "email_cliente": "cliente@example.com",
    "nombre_cliente": "Cliente",
    "telefono_cliente": "099111111",
    "idEstablecimiento": 1,
    "idPersonal": 4,
    "fecha": "2026-11-12",
    "hora": "10:00"
  }
}
```

Tabla de Equivalencias (Consumer → REST API)

Campo en Mensaje MQTT	Atributo en POST /reservas
`turno.nombre_cliente`	`nombre`
`turno.email_cliente`	`email_solicitante`
`turno.telefono_cliente`	`telefono_solicitante`
`turno.idEstablecimiento`	`id_establecimiento`
`turno.idPersonal`	`id_personal`

Campo en Mensaje MQTT	Atributo en POST /reservas
`turno.fecha`	`fecha_turno`
`turno.hora`	`hora_turno`

Nota: El atributo `turno.id` del mensaje enviado por MQTT no representa el ID de la reserva. La API genera y asigna su propio identificador canónico. Cualquier error en los datos de entrada generará una respuesta de error HTTP inmediata, mientras que la validación de disponibilidad de fechas se diferirá a la fase del worker.

Confirmación de Entrega y Reintentos

La suscripción a los mensajes utiliza el nivel de calidad de servicio **QoS 1**, sesión persistente y confirmación explícita (ACK manual):

- **Respuesta exitosa (`202 Accepted`)**: El consumidor confirma (*acknowledge*) el mensaje al broker MQTT para removerlo de la cola.
- **Errores de cliente (`4xx`, excepto `409`)**: Representan datos o referencias inválidos. El consumidor confirma el mensaje para descartarlo y evitar el bloqueo de la cola de procesamiento.
- **Fallas del sistema (`409`, `5xx` o sin respuesta)**: El consumidor **no** confirma el mensaje y procede a reconectarse. Mosquitto reintentará la entrega del mensaje posteriormente.
- **Resiliencia del Generador**: El servicio generador cuenta con su propio esquema de reintentos mientras la API no se encuentre disponible.

La entrega bajo QoS 1 garantiza una semántica de **"al menos una vez"** (*at-least-once*). En caso de que se pierda la confirmación luego de que la API haya registrado la solicitud, el mensaje puede reenviarse. Debido a que el identificador final de la reserva es determinado de forma autónoma por la API, un reenvío generará un nuevo registro de solicitud.

Variables de Configuración

En el entorno de Docker Compose, los contenedores utilizan los siguientes parámetros de configuración:

Variable	Descripción	Valor por Defecto
`API_BASE_URL`	Endpoint base para el consumo de la API REST	`http://localhost:8000`
`API_TIMEOUT_SECONDS`	Tiempo límite para llamadas HTTP (segundos)	`5`
`MQTT_HOST`	Dirección del broker MQTT	`localhost`
`MQTT_PORT`	Puerto de conexión MQTT	`1883`
`MQTT_TOPIC`	Tópico de publicación y suscripción	`turnos/solicitudes`

Dentro del entorno de Docker Compose, ambos contenedores anulan el valor por defecto estableciendo `API_BASE_URL=http://reservas-api:8000`.

- **Parámetros adicionales del Generador**: Permite ajustar `API_RETRY_SECONDS`, `MQTT_RETRY_SECONDS` y `PUBLICATION_INTERVAL_SECONDS`.
- **Parámetros adicionales del Consumidor**: Soporta la definición de `MQTT_CLIENT_ID` y `RECONNECT_SECONDS`.

Despliegue y Diagnóstico

Inicio de Servicios

```
None
docker compose up -d --build
```

Visualización de Logs en Tiempo Real

```
None
docker compose logs -f generador consumidor reservas-api mosquitto
```

Verificación y Puntos de Acceso

- **Documentación Interactiva (API)**: Disponible en `http://localhost:8000/docs`. Desde este portal es posible probar manualmente el endpoint `POST /reservas`, el cual utiliza la misma especificación que el servicio consumidor.
- **Broker MQTT**: Expuesto hacia el host en el puerto `1883`. Entre contenedores de Docker, la comunicación resuelve a través de los nombres de servicio `reservas-api` y `mosquitto`.
- **Aislamiento de Persistencia**: Para garantizar la separación de responsabilidades, los contenedores `generador` y `consumidor` no disponen de credenciales ni accesos a la base de datos MySQL. La herramienta phpMyAdmin se mantiene exclusivamente para tareas administrativas puntuales y no interviene en el pipeline de mensajería.